LAB 15 Integrated capstone: API, offline persistence and multiple screens

Unit 2–5 • 3-hour guided laboratory • CLO-5

Learning goal. Combine remote data, persistent state, Android navigation and error handling into a small deliverable, then plan a larger team semester project against CLO-5.

Core concepts. Integration testing; JSON API + cache + favorites; offline tasks; cross-screen data flow; reproducible Android APK; issue-driven project delivery.

Before you start. Successful completion of Labs 01–14. For Task 1 ensure Android INTERNET permission from Lab 10 is inserted and install packages.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab15
cd mad_lab15
flutter pub add http shared_preferences
```

Solved task 1 | Offline article feed, bookmarks and details

Problem and objective. Build a functioning feed that fetches titles and bodies from JSONPlaceholder, caches the last successful response, retains favorite article IDs and navigates to a detail page. Add Android INTERNET permission as shown in Lab 10.

lib/main.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

class Article {
  final int id;
  final String title;
  final String body;
  Article(this.id, this.title, this.body);
  factory Article.fromJson(Map<String, dynamic> x) => Article(
    x['id'] as int, x['title'] as String, x['body'] as String);
}

void main() => runApp(const MaterialApp(home: ArticleFeed()));

class ArticleFeed extends StatefulWidget {
  const ArticleFeed({super.key});
  @override
  State<ArticleFeed> createState() => _ArticleFeedState();
}

class _ArticleFeedState extends State<ArticleFeed> {
  final prefs = SharedPreferencesAsync();
  List<Article> articles = [];
  Set<String> favorites = {};
  String status = 'Loading...';
  bool busy = false;

  @override
  void initState() { super.initState(); load(); }

  Future<void> load() async {
    if (busy) return;
    setState(() => busy = true);
    var source = 'Live API';
    String raw;
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts'))
          .timeout(const Duration(seconds: 10));
      if (response.statusCode != 200) throw Exception('HTTP error');
```

```
      raw = response.body;
      final parsed = jsonDecode(raw) as List<dynamic>;
      parsed.take(25).map((e) => Article.fromJson(
        e as Map<String, dynamic>)).toList();
      await prefs.setString('feed_cache', raw);
    } catch (_) {
      raw = await prefs.getString('feed_cache') ?? '[]';
      source = raw == '[]' ? 'Offline, no data' : 'Offline cache';
    }
    try {
      final decoded = jsonDecode(raw) as List<dynamic>;
      final values = decoded.take(25).map((e) =>
        Article.fromJson(e as Map<String, dynamic>)).toList();
      final saved = (await prefs.getStringList('favs')) ?? [];
      if (mounted) setState(() {
        articles = values; favorites = saved.toSet(); status = source;
      });
    } catch (_) {
      if (mounted) setState(() => status = 'Invalid response / cache');
    } finally {
      if (mounted) setState(() => busy = false);
    }
  }
```

```
  Future<void> toggle(int id) async {
    final next = {...favorites};
    if (!next.add('$id')) next.remove('$id');
    await prefs.setStringList('favs', next.toList());
    if (mounted) setState(() => favorites = next);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('CSC303 article reader')),
    body: Column(children: [
      ListTile(title: Text(status),
        subtitle: Text('${favorites.length} favorites'),
        trailing: IconButton(onPressed: busy ? null : load,
          icon: const Icon(Icons.refresh))),
      if (busy) const LinearProgressIndicator(),
      Expanded(child: ListView.builder(
        itemCount: articles.length,
        itemBuilder: (context, i) {
          final article = articles[i];
          final saved = favorites.contains('${article.id}');
          return ListTile(
            title: Text(article.title),
            leading: CircleAvatar(child: Text('${article.id}')),
            trailing: IconButton(
              icon: Icon(saved ? Icons.bookmark : Icons.bookmark_border),
              onPressed: () => toggle(article.id)),
            onTap: () => Navigator.push(context, MaterialPageRoute(
              builder: (_) => ArticleDetail(article: article))),
          );
        },
      )),
    ]),
  );
}

class ArticleDetail extends StatelessWidget {
  final Article article;
  const ArticleDetail({super.key, required this.article});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: Text('Article ${article.id}')),
    body: ListView(padding: const EdgeInsets.all(20), children: [
      Text(article.title,
        style: Theme.of(context).textTheme.headlineSmall),
      const SizedBox(height: 12),
      Text(article.body),
    ]),
  );
}
```

How the solution works. The feed retrieves typed data from a public API, writes raw successful responses to cache, then parses and displays them. A set of bookmark IDs is serialized through SharedPreferencesAsync. The details page takes an immutable Article by constructor. Error state and offline/source labels expose whether the content is fresh.

Expected output. Online: 25 articles, bookmarking icons and detail navigation. Offline after a successful refresh: cached articles and existing bookmarks remain.

Check your work. Start online, bookmark two IDs, close app, enable airplane mode, reopen, navigate to a detail and verify two favorites persist.

Solved task 2 | Offline task board with a separate details page

Problem and objective. Create a personal sprint board where tasks can be added and checked off, stored as JSON locally, and opened on a detail route. This has no online dependency and works on a phone immediately after shared_preferences is installed.

lib/main.dart — replace all of Task 1

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class WorkItem {
  String title;
  bool completed;
  WorkItem(this.title, this.completed);
  Map<String, dynamic> toJson() =>
    {'title': title, 'completed': completed};
  factory WorkItem.fromJson(Map<String, dynamic> value) => WorkItem(
    value['title'] as String, value['completed'] as bool);
}

void main() => runApp(const MaterialApp(home: TaskBoard()));

class TaskBoard extends StatefulWidget {
  const TaskBoard({super.key});
  @override
  State<TaskBoard> createState() => _TaskBoardState();
}

class _TaskBoardState extends State<TaskBoard> {
  final prefs = SharedPreferencesAsync();
  final tasks = <WorkItem>[];

  @override
  void initState() { super.initState(); restore(); }

  Future<void> restore() async {
    final text = await prefs.getString('sprint_tasks');
    if (text == null) return;
    try {
      final rows = jsonDecode(text) as List<dynamic>;
      if (mounted) setState(() {
        tasks..clear()..addAll(rows.map((x) =>
          WorkItem.fromJson(x as Map<String, dynamic>)));
      });
    } catch (_) { /* Ignore corrupted demo data. */ }
  }

  Future<void> save() async {
    await prefs.setString('sprint_tasks',
      jsonEncode(tasks.map((x) => x.toJson()).toList()));
  }

  Future<void> addTask() async {
    String draft = '';
    final value = await showDialog<String>(context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('New sprint task'),
        content: TextField(autofocus: true,
```

```
          onChanged: (value) => draft = value,
          decoration: const InputDecoration(hintText: 'Task title')),
        actions: [
          TextButton(onPressed: () => Navigator.pop(dialogContext),
            child: const Text('Cancel')),
          FilledButton(onPressed: () =>
            Navigator.pop(dialogContext, draft.trim()),
            child: const Text('Add')),
        ],
      ));
    if (value == null || value.isEmpty || !mounted) return;
    setState(() => tasks.add(WorkItem(value, false)));
    await save();
  }

  @override
  Widget build(BuildContext context) {
    final finished = tasks.where((x) => x.completed).length;
    return Scaffold(
      appBar: AppBar(title: Text('Sprint board • $finished/${tasks.length}')),
      floatingActionButton: FloatingActionButton(
        onPressed: addTask, child: const Icon(Icons.add)),
      body: ListView.builder(
        itemCount: tasks.length,
        itemBuilder: (context, i) {
          final item = tasks[i];
          return CheckboxListTile(
            value: item.completed,
            title: Text(item.title),
            subtitle: Text(item.completed ? 'Done' : 'Open'),
            secondary: IconButton(
              icon: const Icon(Icons.open_in_new),
              onPressed: () => Navigator.push(context, MaterialPageRoute(
                builder: (_) => TaskDetail(item: item)))),
            onChanged: (v) {
              setState(() => item.completed = v ?? false); save();
            },
          );
        },
      ),
    );
  }
}

class TaskDetail extends StatelessWidget {
  final WorkItem item;
  const TaskDetail({super.key, required this.item});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Task detail')),
    body: Padding(padding: const EdgeInsets.all(20),
      child: Column(crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(item.title,
            style: Theme.of(context).textTheme.headlineSmall),
          Text('Current status: ${item.completed ? 'Done' : 'Open'}'),
        ],
      )),
  );
}
```

How the solution works. Each task is a model object with JSON serialization. The board stores a complete JSON list and restores it when initialized. Checkbox state is changed in memory before persistence. The detail route receives the object and shows current status; this is a small integrated example, not a multi-user synchronized task system.

Expected output. Students can add tasks, complete them, open details and reopen the app to see the saved sprint state.

Check your work. Test a brand-new install, three additions, two completions, force-close, restore and one canceled add dialog.

Final semester project — student brief (50 marks)

Build a medium-sized Android Flutter application for a genuine campus or community need (e.g., timetable and announcements, library reservations, hostel complaint tracking, semester planner, food ordering prototype, or campus shuttle monitoring). The app must be distinguishable from these worked examples and should demonstrate the CDF's CLO-5.

Mandatory requirement	Acceptance evidence
Multiple screens	At least three meaningful screens with coherent navigation and state transfer.
User interface	Responsive and accessible Material UI, form validation, loading/empty/error states.
Online API	At least one meaningful HTTPS endpoint with parsed typed data and error handling.
Persistent storage	SQLite/Drift or Firestore plus an explicit offline or persisted-use case.
Architecture	Recognizable presentation, business logic and repository separation.
Version control	Issue board, feature branches, minimum two reviewed PRs for a team, useful commits.
Verification	Automated tests and documented physical Android testing, flutter analyze passing.
Delivery	GitHub repository, README, report, screens, demo and installable debug/release APK.

Project milestones and demonstration protocol

Proposal: one page containing problem, target users, data model, screen flow, API selection, threat/privacy considerations and issue plan. Sprint 1: navigation and functional UI. Sprint 2: persistence/API and failure handling. Sprint 3: architecture refactoring and automated tests. Final demo: install on a phone from a clean checkout, run the critical journey, deliberately disable networking, show locally retained state and explain the Git contributions of every group member.

Scenario-based practice — unsolved

Scenario 1. Capstone: shuttle tracker. Design a campus shuttle app with live arrivals, saved routes and a useful offline fallback. Specify screen map, source of truth, stale data indicator and fair team ownership.

Scenario 2. Capstone: complaint system. Students submit complaints with optional images and staff update status. Explain access control, upload safety, offline edits, conflict resolution and audit logs.

Technical viva — justify your reasoning

1. What evidence proves an app is offline-capable instead of just remembering UI state in RAM?
2. What data should be the source of truth after reconnecting?
3. How would you prevent duplicate writes when a retry follows a timeout?
4. How do you show ownership and code-review evidence in a three-person project?
5. Which parts of a Flutter app should have unit tests vs integration tests?
6. How should a developer handle migrations after releasing version 1 of an app?

Submission and self-check

Submit a stable final demo, documented Android APK, repo/PR links, tests, API/offline screenshots and the 50-mark project rubric evidence.

CSC303 / NOVEMBER 2026 REVISION

LAB 15 EXTENSION | From worked examples to an assessable app

Scope clarification: Original Task 1 is a network-first article reader with a SharedPreferences cache; Task 2 is a local task board. They are valid practice components, but neither alone fulfills the complete 50-mark final project specification later in Lab 15.

Reference architecture to build for the final project

```
semester_project/
  lib/
    main.dart                    # app, theme, routes
    models/article.dart          # typed data + validation
    data/local/app_database.dart # Drift + schema/migrations
    data/remote/article_api.dart # injectable HTTP client
    data/article_repository.dart # coordinates local/remote
    viewmodels/feed_viewmodel.dart
    screens/feed_screen.dart
    screens/detail_screen.dart
    screens/saved_screen.dart
  test/
    article_repository_test.dart
    feed_viewmodel_test.dart
    widget_test.dart
  README.md
```

Start by integrating the Lab 15 article model and network code with the Lab 09 Drift table. Use a repository to expose a local stream as the screen's data source; keep remote synchronization separate. This is an architectural exercise, not a drop-in replacement for Lab 15's complete `main.dart`.

Data flow and failure handling

```
Android screens -> ViewModel -> Repository
                            |            |
                            v            v
                      Drift/SQLite    HTTPS remote
                            ^            |
                            +-- validated, mapped records

On app start: read local rows immediately.
On refresh: fetch/parse/validate -> database transaction.
On failure: keep local rows; show stale + retry status.
On favorites: save change locally; verify after restart.
```

Integration checkpoints

Stage	Observable outcome
A. UI/navigation	Feed, detail and saved screens; errors/empty/loading states
B. Local persistence	Drift table, insert/update/delete, restart proof
C. Remote import	Typed HTTPS result, timeout/error behavior, safe local update
D. Offline flow	Airplane mode launch shows prior rows; no in-memory illusion
E. Testability	Fake remote/local data tests, ViewModel test and widget test
F. Delivery	Clean checkout, regenerated code, report, two PRs if team

CSC303 / NOVEMBER 2026 REVISION

FINAL PROJECT EXTENSION | Detailed 50-mark rubric

This detailed matrix elaborates the existing page-4 categories while preserving every published category weight. Assessors should score observable evidence rather than screenshots alone.

Category	Marks	Specific evidence
Functionality	15	Core end-to-end user journey (7); three meaningful screens (4); validation, navigation/back actions (4)
API and persistence	10	Meaningful HTTPS parsing and error paths (4); Drift/Firestore persistence (4); offline restart proof (2)
UI and accessibility	7	Responsive layouts (2); large-text/screen-reader checks (2); clear loading/empty/error states (3)
Architecture and tests	6	ViewModel/repository separation (2); meaningful automated tests (3); reproducible clean build (1)
Git process	5	Issue/branch traceability (2); at least two reviewed team PRs (2); contribution explanation (1)
Demo and viva	7	Clean-device critical journey (3); forced-offline demo (2); technical reasoning (2)

Demonstration script and acceptance tests

• From a clean checkout run `flutter pub get`, any required code generation, `flutter analyze`, and project tests. Document generated configuration steps; do not publish secrets.

• On a real Android phone, navigate through three meaningful screens and exercise a primary user action.

• Acquire and display typed data from a real HTTPS service. Show one failed request and an understandable UI message.

• Restart the app with network disabled; show persistent data retrieved from a local database or supported Firestore offline mode, not merely retained in a live widget.

• Open issue/PR history and identify what each teammate contributed and reviewed.

Independent work within a team

Team submissions must include a responsibility matrix and an individual viva. A team's shared functionality does not automatically prove equal participation. Where collaboration is unavailable, the instructor should publish an individual equivalent for the Git-process marks.

Explicit exclusions

The Firebase Storage/FCM demo from Lab 12 is optional unless the chosen project requires it and the instructor has approved service access. Students are not required to incur paid cloud charges to demonstrate CLO-5.